JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ValueRange

java.lang.Object
    java.time.temporal.ValueRange

All Implemented Interfaces:

Serializable

```
public final class ValueRange
extends Object
implements Serializable
```

The range of valid values for a date-time field.

All TemporalField instances have a valid range of values. For example, the ISO day-of-month runs from 1 to somewhere between 28 and 31. This class captures that valid range.

It is important to be aware of the limitations of this class. Only the minimum and maximum values are provided. It is possible for there to be invalid values within the outer range. For example, a weird field may have valid values of 1, 2, 4, 6, 7, thus have a range of '1 - 7', despite the fact that values 3 and 5 are invalid.

Instances of this class are not tied to a specific field.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
int	checkValidIntValue(long value, TemporalField field)	Checks that the specified value is valid and fits in an int.
long	checkValidValue(long value, TemporalField field)	Checks that the specified value is valid.

boolean	**equals**(Object obj)	Checks if this range is equal to another range.
long	**getLargestMinimum**()	Gets the largest possible minimum value that the field can take.
long	**getMaximum**()	Gets the maximum value that the field can take.
long	**getMinimum**()	Gets the minimum value that the field can take.
long	**getSmallestMaximum**()	Gets the smallest possible maximum value that the field can take.
int	**hashCode**()	A hash code for this range.
boolean	**isFixed**()	Is the value range fixed and fully known.
boolean	**isIntValue**()	Checks if all values in the range fit in an `int`.
boolean	**isValidIntValue**(long value)	Checks if the value is within the valid range and that all values in the range fit in an `int`.
boolean	**isValidValue**(long value)	Checks if the value is within the valid range.
static ValueRange	**of**(long min, long max)	Obtains a fixed value range.
static ValueRange	**of**(long min, long maxSmallest, long maxLargest)	Obtains a variable value range.
static ValueRange	**of**(long minSmallest, long minLargest, long maxSmallest, long maxLargest)	Obtains a fully variable value range.
String	**toString**()	

	Outputs this range as a `String`.

Methods declared in class Object

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Method Details

of

```
public static ValueRange of(long min,
                            long max)
```

Obtains a fixed value range.

This factory obtains a range where the minimum and maximum values are fixed. For example, the ISO month-of-year always runs from 1 to 12.

Parameters:

`min` - the minimum value

`max` - the maximum value

Returns:

the ValueRange for min, max, not null

Throws:

`IllegalArgumentException` - if the minimum is greater than the maximum

of

```
public static ValueRange of(long min,
                            long maxSmallest,
                            long maxLargest)
```

Obtains a variable value range.

This factory obtains a range where the minimum value is fixed and the maximum value may vary. For example, the ISO day-of-month always starts at 1, but ends between 28 and 31.

Parameters:

`min` - the minimum value

`maxSmallest` - the smallest maximum value

`maxLargest` - the largest maximum value

Returns:

the ValueRange for min, smallest max, largest max, not null

Throws:

`IllegalArgumentException` - if the minimum is greater than the smallest maximum, or the smallest maximum is greater than the largest maximum

of

```
public static ValueRange of(long minSmallest,
                            long minLargest,
                            long maxSmallest,
                            long maxLargest)
```

Obtains a fully variable value range.

This factory obtains a range where both the minimum and maximum value may vary.

Parameters:

`minSmallest` - the smallest minimum value

`minLargest` - the largest minimum value

`maxSmallest` - the smallest maximum value

`maxLargest` - the largest maximum value

Returns:

the ValueRange for smallest min, largest min, smallest max, largest max, not null

Throws:

`IllegalArgumentException` - if the smallest minimum is greater than the smallest maximum, or the smallest maximum is greater than the largest maximum, or the largest minimum is greater than the largest maximum, or the smallest minimum is greater than the largest minimum

isFixed

```
public boolean isFixed()
```

Is the value range fixed and fully known.

For example, the ISO day-of-month runs from 1 to between 28 and 31. Since there is uncertainty about the maximum value, the range is not fixed. However, for the month of January, the range is always 1 to 31, thus it is fixed.

Returns:

true if the set of values is fixed

getMinimum

```
public long getMinimum()
```

Gets the minimum value that the field can take.

For example, the ISO day-of-month always starts at 1. The minimum is therefore 1.

Returns:

the minimum value for this field

getLargestMinimum

```
public long getLargestMinimum()
```

Gets the largest possible minimum value that the field can take.

For example, the ISO day-of-month always starts at 1. The largest minimum is therefore 1.

Returns:

the largest possible minimum value for this field

getSmallestMaximum

```
public long getSmallestMaximum()
```

Gets the smallest possible maximum value that the field can take.

For example, the ISO day-of-month runs to between 28 and 31 days. The smallest maximum is therefore 28.

Returns:

the smallest possible maximum value for this field

getMaximum

```
public long getMaximum()
```

Gets the maximum value that the field can take.

For example, the ISO day-of-month runs to between 28 and 31 days. The maximum is therefore 31.

Returns:

the maximum value for this field

isIntValue

```
public boolean isIntValue()
```

Checks if all values in the range fit in an `int`.

This checks that all valid values are within the bounds of an `int`.

For example, the ISO month-of-year has values from 1 to 12, which fits in an `int`. By comparison, ISO nano-of-day runs from 1 to 86,400,000,000,000 which does not fit in an `int`.

This implementation uses `getMinimum()` and `getMaximum()`.

Returns:

true if a valid value always fits in an `int`

isValidValue

```
public boolean isValidValue(long value)
```

Checks if the value is within the valid range.

This checks that the value is within the stored range of values.

Parameters:

`value` - the value to check

Returns:

true if the value is valid

isValidIntValue

```
public boolean isValidIntValue(long value)
```

Checks if the value is within the valid range and that all values in the range fit in an `int`.

This method combines `isIntValue()` and `isValidValue(long)`.

Parameters:

`value` - the value to check

Returns:

true if the value is valid and fits in an `int`

checkValidValue

```
public long checkValidValue(long value,
                            TemporalField field)
```

Checks that the specified value is valid.

This validates that the value is within the valid range of values. The field is only used to improve the error message.

Parameters:

`value` - the value to check

`field` - the field being checked, may be null

Returns:

the value that was passed in

See Also:

`isValidValue(long)`

checkValidIntValue

```
public int checkValidIntValue(long value,
                              TemporalField field)
```

Checks that the specified value is valid and fits in an `int`.

This validates that the value is within the valid range of values and that all valid values are within the bounds of an `int`. The field is only used to improve the error message.

Parameters:

`value` - the value to check

`field` - the field being checked, may be null

Returns:

the value that was passed in

See Also:

`isValidIntValue(long)`

equals

```
public boolean equals(Object obj)
```

Checks if this range is equal to another range.

The comparison is based on the four values, minimum, largest minimum, smallest maximum and maximum. Only objects of type `ValueRange` are compared, other types return false.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other range

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this range.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Outputs this range as a `String`.

The format will be '{min}/{largestMin} - {smallestMax}/{max}', where the largestMin or smallestMax sections may be omitted, together with associated slash, if they are the same as the min or max.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this range, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.